香山环境部署验证与操作日志

CIE 香山赛道第一阶段

团队：一拍即合 姓名：王朋志

香山仿真器 emu 构建完成，官方 hello 示例也已编译为 RISC-V 镜像并成功运行。日志打印了参赛标识和部署时的公网 IP，随后出现 HIT GOOD TRAP，退出码为 0。本报告覆盖阶段一的 1.1 环境部署验证和 1.2 操作日志；1.3 向量加法通路分析另行提交。

1.1 环境部署验证

验证程序

在 nexus-am/apps/hello/hello.c 中修改打印字符串。源码直接填入部署时的服务器公网 IP。

```
#include <klib.h>

int main()
{
  printf("hello xiangshan, I am 王朋志, IP address: 121.41.71.206\n");
  return 0;
}
```

运行命令

```
cd "$NOOP_HOME"
set -o pipefail
./build/emu -i "$AM_HOME/apps/hello/build/hello-riscv64-xs.bin" \
 --no-diff -C 2000000 2>&1 | tee ~/stage1-records/11-hello-run.log
printf 'hello simulation exit status: %s\n' "$?"
```

本次运行采用 --no-diff，未启用差分比对。命令设置 2,000,000 周期上限，并用 tee 保存输出；下一条 printf 立即记录该管道的退出状态。

终端输出

```
hello xiangshan, I am 王朋志, IP address: 121.41.71.206
Core 0: HIT GOOD TRAP at pc = 0x8000014c
Core-0 instrCnt = 1,431, cycleCnt = 14,383, IPC = 0.099492
hello simulation exit status: 0
```

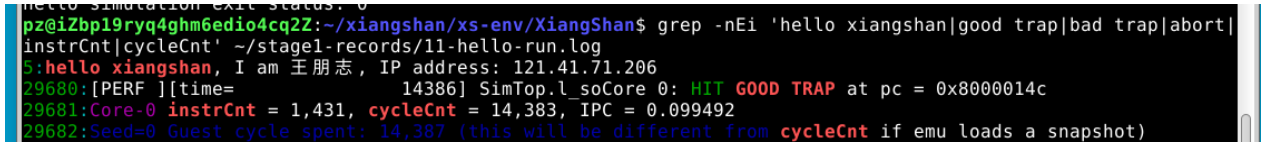


图 1　hello 打印指定信息并出现 GOOD TRAP

记录：hello.c、10-hello-build-retry1.log、11-hello-run.log。构建产物见编译日志，运行结果见图 1 和运行日志。

1.2 操作日志

开发环境与工具版本

表 1 开发环境及工具版本

项目	实际记录
系统与资源	Ubuntu 22.04.5 LTS（x86-64）；nproc：16；内存：61 GiB；系统分区：197G
用户与目录	用户：pz；工作目录：/home/pz/xiangshan/xs-env；日志目录：/home/pz/stage1-records
编译与构建工具	Git 2.34.1；GCC 11.4.0；OpenJDK 和 javac 21.0.12.1；Mill 0.12.3
仿真与 C++工具	Verilator 5.052；构建 emu 使用 Ubuntu clang 14.0.0
RISC-V 工具链	riscv64-linux-gnu-gcc；已安装对应的 gcc、g++ 和 binutils 交叉工具包

日志采集与时间记录

root 会话从 11:27 开始记录，pz 会话从 11:44 开始记录。emu 产物的修改时间为 14:02，hello 验证截图记录于 15:13。交互过程用 script 记录，构建和运行输出用 tee 保存，长任务在 tmux 的 xs-build 会话中执行。

```
mkdir -p ~/stage1-records
script -a ~/stage1-records/02-xiangshan-deployment.log
set -o pipefail
```

新终端进入 xs-env 后执行 source ./env.sh，使 NOOP_HOME、NEMU_HOME 和 AM_HOME 分别指向 XiangShan、NEMU 和 nexus-am。原始记录保留了交叉工具链前缀，未单独记录其精确版本。

记录：01-cloud-deployment.log、02-xiangshan-deployment.log、05-emu-build-session.log、xs-env-version.txt。

部署步骤

1 配置远程桌面与开发用户

先以 root 安装 XFCE 和 xrdp，再创建 pz 用户并将其加入 sudo 组。Windows 经 SSH 本地端口转发连接远程桌面。部署日志中 xrdp 和 xrdp-sesman 均为 active。

```
apt-get install -y xfce4 xfce4-terminal xrdp xorgxrdp dbus-x11
adduser pz
usermod -aG sudo pz
```

2 获取源码与安装工具

```
sudo apt-get update
sudo apt-get install -y git ca-certificates
mkdir -p ~/xiangshan
cd ~/xiangshan
git clone https://github.com/OpenXiangShan/xs-env.git
cd xs-env
sudo bash ./setup-tools.sh 2>&1 | tee ~/stage1-records/03-setup-tools.log
source ./setup.sh
source ./env.sh
git submodule status --recursive
```

setup-tools.sh 和 setup.sh 均返回 0。后续以 pz 构建时仍找不到 java 和 clang++，按步骤 4 补齐依赖。

3 构建 NEMU 参考模型

```
cd "$NEMU_HOME"
make riscv64-xs-ref_defconfig
make -j8 2>&1 | tee ~/stage1-records/04-nemu-build.log
```

构建日志显示，链接生成了 NEMU/build/riscv64-nemu-interpreter-so。这个共享库在宿主机上充当参考模型；步骤 6 生成的 RISC-V 镜像由香山仿真器执行。

4 补齐构建依赖

```
sudo apt-get install -y openjdk-21-jdk-headless
sudo apt-get install -y clang
java -version
javac -version
clang++ --version
```

补装后，java 和 javac 均报告 21.0.12.1，clang 报告 14.0.0。随后的重试完成了 emu 构建。

构建仿真器与测试镜像

5 构建香山 emu

```
cd "$NOOP_HOME"
make emu EMU_TRACE=fst -j8 2>&1 | \
  tee ~/stage1-records/06-emu-build-retry4.log
ls -lh build/emu
file build/emu
./build/emu --help 2>&1 | tee ~/stage1-records/07-emu-help.log
```

以 DefaultConfig 构建单核 top.SimTop，使用 -j8 和 EMU_TRACE=fst。构建退出码为 0，生成的 build/emu 为 261M 的 x86-64 ELF 文件，--help 可正常执行。本次 hello 运行未验证 FST 波形导出。

6 编译并运行 hello

```
sudo apt install gcc-riscv64-linux-gnu \
  g++-riscv64-linux-gnu binutils-riscv64-linux-gnu
cd "$AM_HOME/apps/hello"
make ARCH=riscv64-xs CROSS_COMPILE=riscv64-linux-gnu- 2>&1 | \
  tee ~/stage1-records/10-hello-build-retry1.log
```

构建日志记录了 hello.c、AM 和 klib 的编译，产物为 build/hello-riscv64-xs.elf 和 .bin。按 1.1 节的命令加载 .bin，得到预期打印、GOOD TRAP 和退出码 0。

故障与处理速查

表 2　构建故障与处理结果

故障	处理	结果与记录
在 NEMU 目录执行 make emu	切换到 $NOOP_HOME	06-emu-build.log 记录目标不存在
exec: java: not found	安装 OpenJDK 21 并核对版本	06-emu-build-retry1.log 记录报错；补装后开始生成 RTL
找不到 clang++	安装 clang 后重新构建	retry3 失败；retry4 与会话日志确认构建成功
交叉编译器名称不匹配	指定 CROSS_COMPILE=riscv64-linux-gnu-	10-hello-build.log 记录失败；retry1 生成镜像

编译选项和宏重定义警告保留在原始日志中。本次 emu 构建及 hello 验证均完成。

记录：06 系列构建日志、07-emu-help.log、10 系列编译日志、11-hello-run.log、05-emu-build-session.log。

构建故障与修复记录

以下四类故障按发生顺序排列。每项保留原始报错、修复操作和重试结果。

1 在 NEMU 目录执行 emu 构建

```
失败命令  make emu EMU_TRACE=fst -j8
实际报错  make: *** No rule to make target 'emu'.  Stop.
```

NEMU 的 Makefile 没有 emu 目标。切换到 $NOOP_HOME 后重跑，构建开始调用 Mill 生成 RTL，随后因缺少 Java 停止。

记录：06-emu-build.log、05-emu-build-session.log。

2 Mill 找不到 Java

```
实际报错  /home/pz/.cache/mill/download/0.12.3: 16: exec: java: not found
        make: *** [Makefile:224: build/rtl/SimTop.sv] Error 127
修复命令  sudo apt-get install -y openjdk-21-jdk-headless
```

Mill 找不到 java，RTL 尚未生成。补装后，java 和 javac 均返回 21.0.12.1；后续日志进入 RTL 生成和 Verilator 模型编译。

记录：06-emu-build-retry1.log、05-emu-build-session.log。

3 模型编译找不到 clang++

```
实际报错  make[3]: clang++: No such file or directory
        make: *** [Makefile:277: emu] Error 2
修复命令  sudo apt-get install -y clang
```

此时 RTL 已生成，构建停在 C++ 模型编译。安装 clang 后重跑 make emu，最终日志记录 emu build exit status: 0。生成的 build/emu 为 261M，--help 可正常执行。

记录：06-emu-build-retry3.log、06-emu-build-retry4.log、05-emu-build-session.log。

4 交叉编译器前缀不匹配

```
失败命令  make ARCH=riscv64-xs
实际报错  make: riscv64-unknown-linux-gnu-gcc: No such file or directory
修复参数  CROSS_COMPILE=riscv64-linux-gnu-
```

hello 默认查找 riscv64-unknown-linux-gnu-gcc，而安装的 Ubuntu 工具链使用 riscv64-linux-gnu-。指定前缀后，LD 和 OBJCOPY 生成了 .elf 与 .bin；随后运行 .bin，得到 GOOD TRAP 和退出码 0。

记录：10-hello-build.log、10-hello-build-retry1.log、11-hello-run.log、图 1。

原始日志索引

随报告提交的 stage1-records-upload.tar.gz 包含 16 份日志或版本记录及 hello.c。下表按部署、构建和运行顺序列出各文件。

表 3 原始日志与记录内容

文件或文件组	记录内容
01-cloud-deployment.log	初始资源检查、用户创建和远程桌面配置
02-xiangshan-deployment.log	pz 部署主会话、源码和子模块初始化
03-setup-tools.log	工具安装脚本输出
04-nemu-build.log	NEMU 编译和链接
05-emu-build-session.log	emu 构建会话、依赖补装和退出状态
06-emu-build.log 06-emu-build-retry1.log 06-emu-build-retry3.log 06-emu-build-retry4.log	依次记录目录错误、Java 缺失、clang++ 缺失及最终构建成功
07-emu-help.log	仿真器启动及可用选项
08-coremark-run.log 09-coremark-bundled-ref.log	两次 CoreMark 差分尝试：前者状态不一致，后者被信号中断
10-hello-build.log 10-hello-build-retry1.log	交叉编译失败；修正前缀后生成镜像
11-hello-run.log	1.1 节的验证输出、正常结束标志和运行统计
xs-env-version.txt hello.c	各仓库提交号与最终验证源码

参考教程

部署与 hello 验证参考 XiangShanLab 开发环境教程第 1 章的 1.5.1—1.5.4 节。本次使用 DefaultConfig，编译 hello 时显式指定交叉编译前缀，运行时加了周期上限并保存日志。

OpenXiangShan XiangShanLab 开发环境第一章